

1. Single Responsibility Principle (SRP)

Single Responsibility Principle podrazumijeva da svaka klasa ima jasno definisanu odgovornost unutar sistema.

U datom dijagramu moguće je uočiti da su određene klase fokusirane na specifične domenske funkcionalnosti:

- klasa **Termin** upravlja životnim ciklusom termina (rezervacija, potvrda, otkazivanje),
- klasa **Recenzija** je zadužena za rad sa ocjenama i komentarima,
- klasa **Transakcija** obrađuje finansijske operacije i stanje plaćanja,
- klasa **Obavjestenje** upravlja slanjem i statusom poruka korisnicima.

Ovakva podjela omogućava jasnu separaciju odgovornosti između različitih dijelova sistema i olakšava održavanje.

2. Open/Closed Principle (OCP)

Open/Closed Principle nalaže da sistem bude otvoren za proširenje, ali zatvoren za izmjene postojećeg koda.

Dijagram pokazuje korištenje enumeracija kao što su:

- **TipInstrukcija,**
- **StatusTermina,**
- **NacinPlacanja,**
- **StatusPlacanja,**
- **Uloga.**

Ove enumeracije omogućavaju proširenje sistema dodavanjem novih vrijednosti bez potrebe za promjenom osnovne strukture klasa. Na taj način, ponašanje sistema se može proširivati kroz nove statuse, tipove ili uloge.

Također, jasno definisane metode unutar klasa omogućavaju dodavanje novih funkcionalnosti kroz proširenje postojećih komponenti.

3. Liskov Substitution Principle (LSP)

Liskov Substitution Principle odnosi se na mogućnost da se objekti jednog tipa zamijene objektima drugog tipa bez narušavanja ispravnosti sistema.

U ovom dijagramu, princip je podržan kroz jasno definisane entitete i njihove uloge u sistemu. Sve klase komuniciraju putem stabilnih i očekivanih interfejsa (metoda), što omogućava konzistentno ponašanje objekata unutar sistema.

Na primjer, operacije nad klasama kao što su **Termin**, **Transakcija** i **Recenzija** su definisane na način da njihovo ponašanje ostaje predvidivo bez obzira na kontekst u kojem se koriste.

4. Interface Segregation Principle (ISP)

Interface Segregation Principle podrazumijeva da klase ne trebaju zavisiti od metoda koje ne koriste, odnosno da interfejsi trebaju biti usko specijalizirani.

U dijagramu se može primijetiti da su funkcionalnosti raspoređene kroz više klasa koje predstavljaju različite domenske cjeline. Umjesto jednog velikog interfejsa, sistem koristi više manjih skupova metoda raspoređenih po klasama:

- operacije vezane za termine nalaze se u klasi **Termin**,
- operacije vezane za recenzije u klasi **Recenzija**,
- operacije vezane za transakcije u klasi **Transakcija**.

Ovakav pristup omogućava da svaka klasa koristi samo one metode koje su joj potrebne.

5. Dependency Inversion Principle (DIP)

Dependency Inversion Principle nalaže da visokonivojske komponente ne zavise od niskonivojskih implementacija, već od apstrakcija.

U datom dijagramu, zavisnosti između klasa ostvarene su preko jasno definisanih odnosa i tipova podataka. Korištenje apstraktnih koncepata kao što su enumeracije i međusobne relacije između klasa omogućava slabiju spregnutost sistema.

Na primjer:

- klasa **Termin** koristi apstraktne tipove kao što su **StatusTermina** i **TipInstrukcija**,
- klasa **Transakcija** koristi **StatusPlacanja** i **NacinPlacanja**,
- klasa **Recenzija** komunicira sa korisnicima kroz identifikatore i reference.

Ovakav dizajn doprinosi fleksibilnosti sistema i olakšava eventualne promjene implementacije bez utjecaja na ostatak sistema.